

Step 1: Setup & Install Dependencies

```
!pip install torchaudio librosa jiwer
```

```
Requirement already satisfied: sympy==1.13.1 in /usr/local/lib/python3.11/dist-packages (from torch==2.6.0->torchaudio) (1.13.1)
Requirement already satisfied: mpmath<1.4,>=1.1.0 in /usr/local/lib/python3.11/dist-packages (from sympy==1.13.1->torch==2.6.0->torch) (1.3.0)
Requirement already satisfied: audioread>=2.1.9 in /usr/local/lib/python3.11/dist-packages (from librosa) (3.0.1)
Requirement already satisfied: numba>=0.51.0 in /usr/local/lib/python3.11/dist-packages (from librosa) (0.60.0)
Requirement already satisfied: numpy>=1.22.3 in /usr/local/lib/python3.11/dist-packages (from librosa) (2.0.2)
Requirement already satisfied: scipy>=1.6.0 in /usr/local/lib/python3.11/dist-packages (from librosa) (1.14.1)
Requirement already satisfied: scikit-learn>=1.1.0 in /usr/local/lib/python3.11/dist-packages (from librosa) (1.6.1)
Requirement already satisfied: joblib>=1.0 in /usr/local/lib/python3.11/dist-packages (from librosa) (1.4.2)
Requirement already satisfied: decorator>=4.3.0 in /usr/local/lib/python3.11/dist-packages (from librosa) (4.4.2)
Requirement already satisfied: soundfile>=0.12.1 in /usr/local/lib/python3.11/dist-packages (from librosa) (0.13.1)
Requirement already satisfied: pooch>=1.1 in /usr/local/lib/python3.11/dist-packages (from librosa) (1.8.2)
Requirement already satisfied: soxr>=0.3.2 in /usr/local/lib/python3.11/dist-packages (from librosa) (0.5.0.post1)
Requirement already satisfied: lazy_loader>=0.1 in /usr/local/lib/python3.11/dist-packages (from librosa) (0.4)
Requirement already satisfied: msgpack>=1.0 in /usr/local/lib/python3.11/dist-packages (from librosa) (1.1.0)
Requirement already satisfied: click>=8.1.8 in /usr/local/lib/python3.11/dist-packages (from jiwer) (8.1.8)
Collecting rapidfuzz>=3.9.7 (from jiwer)
  Downloading rapidfuzz-3.13.0-cp311-cp311-manylinux_2_17_x86_64.manylinux2014_x86_64.whl.metadata (12 kB)
Requirement already satisfied: packaging in /usr/local/lib/python3.11/dist-packages (from lazy_loader>=0.1->librosa) (24.2)
Requirement already satisfied: llvmlite<0.44,>=0.43.0dev0 in /usr/local/lib/python3.11/dist-packages (from numba>=0.51.0->librosa) (0.43.0)
Requirement already satisfied: platformdirs>=2.5.0 in /usr/local/lib/python3.11/dist-packages (from pooch>=1.1->librosa) (4.3.7)
Requirement already satisfied: requests>=2.19.0 in /usr/local/lib/python3.11/dist-packages (from pooch>=1.1->librosa) (2.32.3)
Requirement already satisfied: threadpoolctl>=3.1.0 in /usr/local/lib/python3.11/dist-packages (from scikit-learn>=1.1.0->librosa) (3.5.0)
Requirement already satisfied: cffi>=1.0 in /usr/local/lib/python3.11/dist-packages (from soundfile>=0.12.1->librosa) (1.17.1)
Requirement already satisfied: pycparser in /usr/local/lib/python3.11/dist-packages (from cffi>=1.0->soundfile>=0.12.1->librosa) (2.23.0)
Requirement already satisfied: charset-normalizer<4,>=2 in /usr/local/lib/python3.11/dist-packages (from requests>=2.19.0->pooch>=1.1->librosa) (3.4.0)
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.11/dist-packages (from requests>=2.19.0->pooch>=1.1->librosa) (3.10.1)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.11/dist-packages (from requests>=2.19.0->pooch>=1.1->librosa) (2.2.3)
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.11/dist-packages (from requests>=2.19.0->pooch>=1.1->librosa) (2025.1.1)
Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.11/dist-packages (from jinja2->torch==2.6.0->torchaudio) (3.0.2)
Downloading nvidia_cublas_cu12-12.4.5.8-py3-none-manylinux2014_x86_64.whl (363.4 MB)
 363.4/363.4 MB 4.1 MB/s eta 0:00:00
Downloading nvidia_cuda_cupti_cu12-12.4.127-py3-none-manylinux2014_x86_64.whl (13.8 MB)
 13.8/13.8 MB 35.8 MB/s eta 0:00:00
Downloading nvidia_cuda_nvrtc_cu12-12.4.127-py3-none-manylinux2014_x86_64.whl (24.6 MB)
 24.6/24.6 MB 35.6 MB/s eta 0:00:00
Downloading nvidia_cuda_runtime_cu12-12.4.127-py3-none-manylinux2014_x86_64.whl (883 kB)
 883.7/883.7 kB 21.2 MB/s eta 0:00:00
Downloading nvidia_cudnn_cu12-9.1.0.70-py3-none-manylinux2014_x86_64.whl (664.8 MB)
 380.1/664.8 MB 3.9 MB/s eta 0:01:14
Downloading nvidia_cufft_cu12-11.2.1.3-py3-none-manylinux2014_x86_64.whl (211.5 MB)
 211.5/211.5 MB 5.1 MB/s eta 0:00:00
Downloading nvidia_curand_cu12-10.3.5.147-py3-none-manylinux2014_x86_64.whl (56.3 MB)
 56.3/56.3 MB 10.9 MB/s eta 0:00:00
Downloading nvidia_cusolver_cu12-11.6.1.9-py3-none-manylinux2014_x86_64.whl (127.9 MB)
 127.9/127.9 MB 7.4 MB/s eta 0:00:00
Downloading nvidia_cusparses_cu12-12.3.1.170-py3-none-manylinux2014_x86_64.whl (207.5 MB)
 207.5/207.5 MB 6.3 MB/s eta 0:00:00
Downloading nvidia_nvjitlink_cu12-12.4.127-py3-none-manylinux2014_x86_64.whl (21.1 MB)
 21.1/21.1 MB 97.2 MB/s eta 0:00:00
Downloading jiwer-3.1.0-py3-none-any.whl (22 kB)
Downloading rapidfuzz-3.13.0-cp311-cp311-manylinux_2_17_x86_64.manylinux2014_x86_64.whl (3.1 MB)
 3.1/3.1 MB 94.9 MB/s eta 0:00:00
ERROR: THESE PACKAGES DO NOT MATCH THE HASHES FROM THE REQUIREMENTS FILE. If you have updated the package versions, please update the
unknown package:
  Expected sha256 165764f44ef8c61fcd9f769d687e06374059fbb388b6c89ecb0e28793a6f
  Got            88fb5fa2cee69e0553886dbd719fe7c68f3b083d3885ffbd483507a4f1419edf
```

Step 2: Mount Google Drive

```
from google.colab import drive
drive.mount('/content/drive')
```

Mounted at /content/drive

Step 3: Import Libraries & Define Constants

```

import os
import torch
import torchaudio
import torchaudio.transforms as T
from torch.utils.data import Dataset, DataLoader
import torch.nn as nn
import torch.nn.functional as F
import matplotlib.pyplot as plt
import IPython.display as ipd

# Constants
SAMPLE_RATE = 16000
N_MELS = 128
MAX_SAMPLES = 1000

import torchaudio
import torchaudio.transforms as T
import matplotlib.pyplot as plt
import torch

# Load an example waveform from torchaudio (or replace with your own)
waveform, SAMPLE_RATE = torchaudio.load(torchaudio.utils.download_asset("tutorial-assets/steam-train-whistle-daniel_simon.wav"))

# Define MelSpectrogram transform
transform = T.MelSpectrogram(
    sample_rate=SAMPLE_RATE,
    n_mels=128,
    n_fft=2048,
    hop_length=512
)

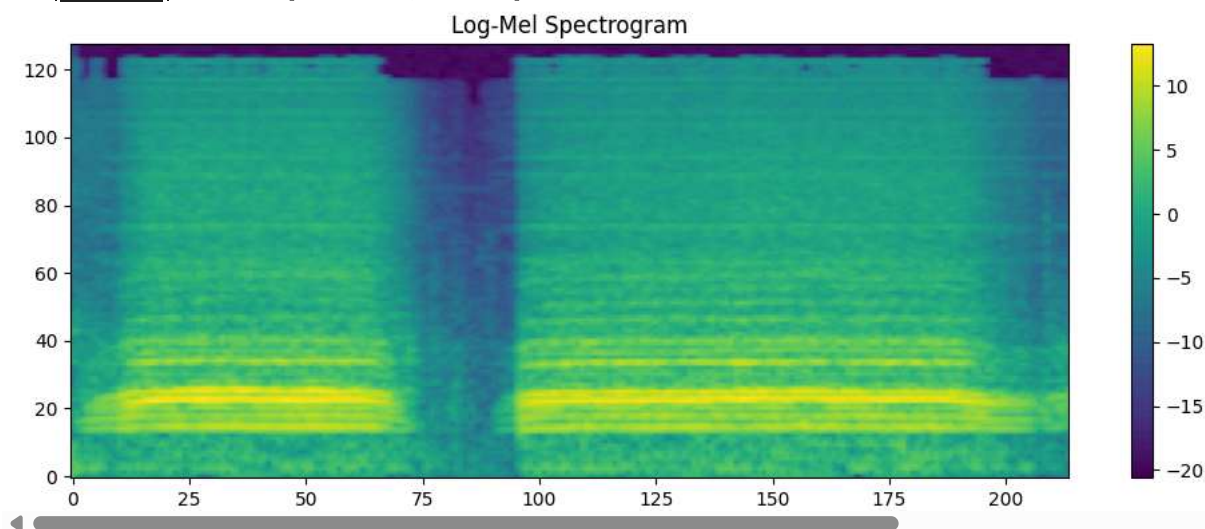
# Apply the transform to the waveform
mel_spec = transform(waveform)

# Convert to log scale for better visualization
log_mel_spec = mel_spec[0].log2().numpy()

# Plot the log-mel spectrogram
plt.figure(figsize=(10, 4))
plt.imshow(log_mel_spec, aspect='auto', origin='lower')
plt.title('Log-Mel Spectrogram')
plt.colorbar()
plt.tight_layout()
plt.show()

```

100% | 427k/427k [00:00<00:00, 59.5MB/s]



✓ Step 4: Helper Functions for Transcript Processing

```

def parse_transcripts(data_dir):
    transcript_dict = {}
    for root, _, files in os.walk(data_dir):
        for file in files:
            if file.endswith(".trans.txt"):
                with open(os.path.join(root, file), "r") as f:
                    for line in f:
                        parts = line.strip().split()
                        file_id = parts[0]
                        transcript = " ".join(parts[1:]).lower()
                        transcript_dict[file_id] = transcript
    return transcript_dict

def build_vocab(transcript_dict):
    all_text = " ".join(transcript_dict.values())
    chars = sorted(set(all_text))
    chars = ['<pad>', '<sos>', '<eos>'] + list(chars)
    char2idx = {ch: idx for idx, ch in enumerate(chars)}
    idx2char = {idx: ch for ch, idx in char2idx.items()}
    return char2idx, idx2char

def text_to_indices(text, char2idx):
    return [char2idx["<sos>"]] + [char2idx[ch] for ch in text if ch in char2idx] + [char2idx["<eos>"]]

```

✓ Step 5: Dataset & Collate Function

```

class LibriSpeechDataset(Dataset):
    def __init__(self, data_dir, transcripts, char2idx, max_samples=None):
        self.data = []
        self.char2idx = char2idx
        self.mel_transform = T.MelSpectrogram(sample_rate=SAMPLE_RATE, n_mels=N_MELS)
        count = 0
        for root, _, files in os.walk(data_dir):
            for file in files:
                if file.endswith(".flac"):
                    file_id = os.path.splitext(file)[0]
                    if file_id in transcripts:
                        self.data.append((os.path.join(root, file), transcripts[file_id]))
                        count += 1
                        if max_samples and count >= max_samples:
                            break
            if max_samples and count >= max_samples:
                break

    def __len__(self):
        return len(self.data)

    def __getitem__(self, idx):
        file_path, text = self.data[idx]
        waveform, sr = torchaudio.load(file_path)
        if sr != SAMPLE_RATE:
            resample = T.Resample(sr, SAMPLE_RATE)
            waveform = resample(waveform)
        mel = self.mel_transform(waveform).squeeze(0).transpose(0, 1)
        label = torch.tensor(text_to_indices(text, self.char2idx))
        return mel, label

    def collate_fn(batch):
        mels, labels = zip(*batch)
        mel_lengths = [mel.shape[0] for mel in mels]
        label_lengths = [len(label) for label in labels]
        mel_padded = nn.utils.rnn.pad_sequence(mels, batch_first=True)
        label_padded = nn.utils.rnn.pad_sequence(labels, batch_first=True)
        return mel_padded, label_padded, mel_lengths, label_lengths

```

✓ Step 6 : Load Dataset and Inspect One Batch

```
DATA_DIR = "/content/drive/MyDrive/train-clean-5"
```

```
# Load and prepare dataset
transcripts = parse_transcripts(DATA_DIR)
char2idx, idx2char = build_vocab(transcripts)
dataset = LibriSpeechDataset(DATA_DIR, transcripts, char2idx, max_samples=MAX_SAMPLES)
dataloader = DataLoader(dataset, batch_size=4, shuffle=True, collate_fn=collate_fn)
```

```
# Check one batch
for mels, labels, mel_lengths, label_lengths in dataloader:
    print(" Mel batch shape:", mels.shape)
    print("Label batch shape:", labels.shape)
    break
```

```
↗ Mel batch shape: torch.Size([4, 1345, 128])
Label batch shape: torch.Size([4, 297])
```

✓ Step 7: LAS Model Architecture

```
# Encoder: Bidirectional LSTM
class Encoder(nn.Module):
    def __init__(self, input_dim, hidden_dim):
        super(Encoder, self).__init__()
        self.lstm = nn.LSTM(input_dim, hidden_dim, batch_first=True, bidirectional=True)

    def forward(self, x):
        outputs, _ = self.lstm(x) # outputs: (batch, time, hidden*2)
        return outputs

# Attention Mechanism
class Attention(nn.Module):
    def __init__(self, encoder_hidden, decoder_hidden):
        super(Attention, self).__init__()
        self.attn = nn.Linear(encoder_hidden * 2 + decoder_hidden, decoder_hidden)
        self.v = nn.Parameter(torch.rand(decoder_hidden))

    def forward(self, hidden, encoder_outputs):
        timestep = encoder_outputs.size(1)
        h = hidden.unsqueeze(1).repeat(1, timestep, 1)
        energy = torch.tanh(self.attn(torch.cat((h, encoder_outputs), dim=2))) # (B, T, D)
        energy = energy.transpose(1, 2) # (B, D, T)
        v = self.v.repeat(encoder_outputs.size(0), 1).unsqueeze(1) # (B, 1, D)
        attn_weights = torch.bmm(v, energy).squeeze(1) # (B, T)
        return F.softmax(attn_weights, dim=1) # (B, T)

# Decoder: Uses attention to decode step-by-step
class Decoder(nn.Module):
    def __init__(self, encoder_hidden, decoder_hidden, output_dim):
        super(Decoder, self).__init__()
        self.embedding = nn.Embedding(output_dim, decoder_hidden)
        self.lstm = nn.LSTM(encoder_hidden * 2 + decoder_hidden, decoder_hidden, batch_first=True)
        self.fc = nn.Linear(encoder_hidden * 2 + decoder_hidden, output_dim)
        self.attention = Attention(encoder_hidden, decoder_hidden)

    def forward(self, encoder_outputs, targets=None, teacher_forcing_ratio=0.5):
        batch_size = encoder_outputs.size(0)
        max_len = 50 # Can be dynamic
        device = encoder_outputs.device
        output_dim = self.fc.out_features

        outputs = []
        input_token = torch.full((batch_size,), 1, dtype=torch.long, device=device) # <sos>

        hidden = torch.zeros(1, batch_size, encoder_outputs.size(2) // 2).to(device)
        cell = torch.zeros(1, batch_size, encoder_outputs.size(2) // 2).to(device)

        for t in range(max_len):
            embedded = self.embedding(input_token).unsqueeze(1) # (B, 1, D)
            attn_weights = self.attention(hidden.squeeze(0), encoder_outputs) # (B, T)
            context = torch.bmm(attn_weights.unsqueeze(1), encoder_outputs) # (B, 1, H*2)
            rnn_input = torch.cat((embedded, context), dim=2) # (B, 1, H*2+D)
```

```

output, (hidden, cell) = self.lstm(rnn_input, (hidden, cell))
prediction = self.fc(torch.cat((output.squeeze(1), context.squeeze(1)), dim=1)) # (B, V)
outputs.append(prediction.unsqueeze(1)) # collect

if targets is not None and torch.rand(1).item() < teacher_forcing_ratio:
    input_token = targets[:, t] if t < targets.size(1) else input_token
else:
    input_token = prediction.argmax(1)

return torch.cat(outputs, dim=1) # (B, max_len, V)

```

```

# LAS Model (Wrapper)
class LAS(nn.Module):
    def __init__(self, input_dim, encoder_hidden, decoder_hidden, output_dim):
        super(LAS, self).__init__()
        self.encoder = Encoder(input_dim, encoder_hidden)
        self.decoder = Decoder(encoder_hidden, decoder_hidden, output_dim)

    def forward(self, x, targets=None):
        encoder_outputs = self.encoder(x)
        return self.decoder(encoder_outputs, targets)

```

✓ Step 8: Training Setup

```

import torch.optim as optim

# Model parameters
encoder_hidden = 256
decoder_hidden = 256
output_dim = len(char2idx)
input_dim = N_MELS # each timestep in mel spectrogram

# Model, optimizer, loss
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model = LAS(input_dim, encoder_hidden, decoder_hidden, output_dim).to(device)

criterion = nn.CrossEntropyLoss(ignore_index=char2idx["<pad>"])
optimizer = optim.Adam(model.parameters(), lr=0.001)

```

✓ Step 9: Training Loop

```

def train(model, dataloader, criterion, optimizer, device, num_epochs=5):
    model.train()
    for epoch in range(num_epochs):
        total_loss = 0
        for mels, labels, mel_lengths, label_lengths in dataloader:
            mels = mels.to(device)
            labels = labels.to(device)

            optimizer.zero_grad()
            outputs = model(mels, labels) # (B, T_pred, V)

            # Align lengths
            T_pred = outputs.size(1)
            T_true = labels.size(1)

            if T_pred < T_true:
                labels = labels[:, :T_pred]
            elif T_pred > T_true:
                pad_value = char2idx["<pad>"]
                padding = torch.full((labels.size(0), T_pred - T_true), pad_value, dtype=torch.long).to(device)
                labels = torch.cat([labels, padding], dim=1)

            # Use reshape instead of view
            outputs = outputs.reshape(-1, outputs.size(2)) # (B*T, V)
            labels = labels.reshape(-1) # (B*T)

```

```

        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()

    total_loss += loss.item()

    print(f"Epoch {epoch + 1}/{num_epochs} | Loss: {total_loss / len(dataloader):.4f}")

```

✓ Step 10: Run Training

```
train(model, dataloader, criterion, optimizer, device, num_epochs=50)
```

```

Epoch 1/50 | Loss: 2.6126
Epoch 2/50 | Loss: 2.5061
Epoch 3/50 | Loss: 2.4610
Epoch 4/50 | Loss: 2.4512
Epoch 5/50 | Loss: 2.4181
Epoch 6/50 | Loss: 2.4053
Epoch 7/50 | Loss: 2.4082
Epoch 8/50 | Loss: 2.3971
Epoch 9/50 | Loss: 2.3830
Epoch 10/50 | Loss: 2.3700
Epoch 11/50 | Loss: 2.3630
Epoch 12/50 | Loss: 2.3492
Epoch 13/50 | Loss: 2.3338
Epoch 14/50 | Loss: 2.3164
Epoch 15/50 | Loss: 2.2970
Epoch 16/50 | Loss: 2.2753
Epoch 17/50 | Loss: 2.2468
Epoch 18/50 | Loss: 2.2317
Epoch 19/50 | Loss: 2.2125
Epoch 20/50 | Loss: 2.1745
Epoch 21/50 | Loss: 2.1433
Epoch 22/50 | Loss: 2.1203
Epoch 23/50 | Loss: 2.0654
Epoch 24/50 | Loss: 2.0468
Epoch 25/50 | Loss: 2.0254
Epoch 26/50 | Loss: 1.9986
Epoch 27/50 | Loss: 1.9535
Epoch 28/50 | Loss: 1.9312
Epoch 29/50 | Loss: 1.9010
Epoch 30/50 | Loss: 1.8885
Epoch 31/50 | Loss: 1.8398
Epoch 32/50 | Loss: 1.8036
Epoch 33/50 | Loss: 1.7739
Epoch 34/50 | Loss: 1.7506
Epoch 35/50 | Loss: 1.7120
Epoch 36/50 | Loss: 1.6883
Epoch 37/50 | Loss: 1.6331
Epoch 38/50 | Loss: 1.5968
Epoch 39/50 | Loss: 1.5510
Epoch 40/50 | Loss: 1.5193
Epoch 41/50 | Loss: 1.4927
Epoch 42/50 | Loss: 1.4663
Epoch 43/50 | Loss: 1.4079
Epoch 44/50 | Loss: 1.3463
Epoch 45/50 | Loss: 1.3424
Epoch 46/50 | Loss: 1.2957
Epoch 47/50 | Loss: 1.2718
Epoch 48/50 | Loss: 1.1796
Epoch 49/50 | Loss: 1.1575
Epoch 50/50 | Loss: 1.1229

```

```

epoch_losses = [
    2.6126, 2.5061, 2.4610, 2.4512, 2.4181, 2.4053, 2.4082, 2.3971, 2.3830,
    2.3700, 2.3630, 2.3492, 2.3338, 2.3164, 2.2970, 2.2753, 2.2468, 2.2317,
    2.2125, 2.1745, 2.1433, 2.1203, 2.0654, 2.0468, 2.0254, 1.9986, 1.9535,
    1.9312, 1.9010, 1.8885, 1.8398, 1.8036, 1.7739, 1.7506, 1.7120, 1.6883,
    1.6331, 1.5968, 1.5510, 1.5193, 1.4927, 1.4663, 1.4079 ]

```

```
import numpy as np
```

```
batch_losses = []
```

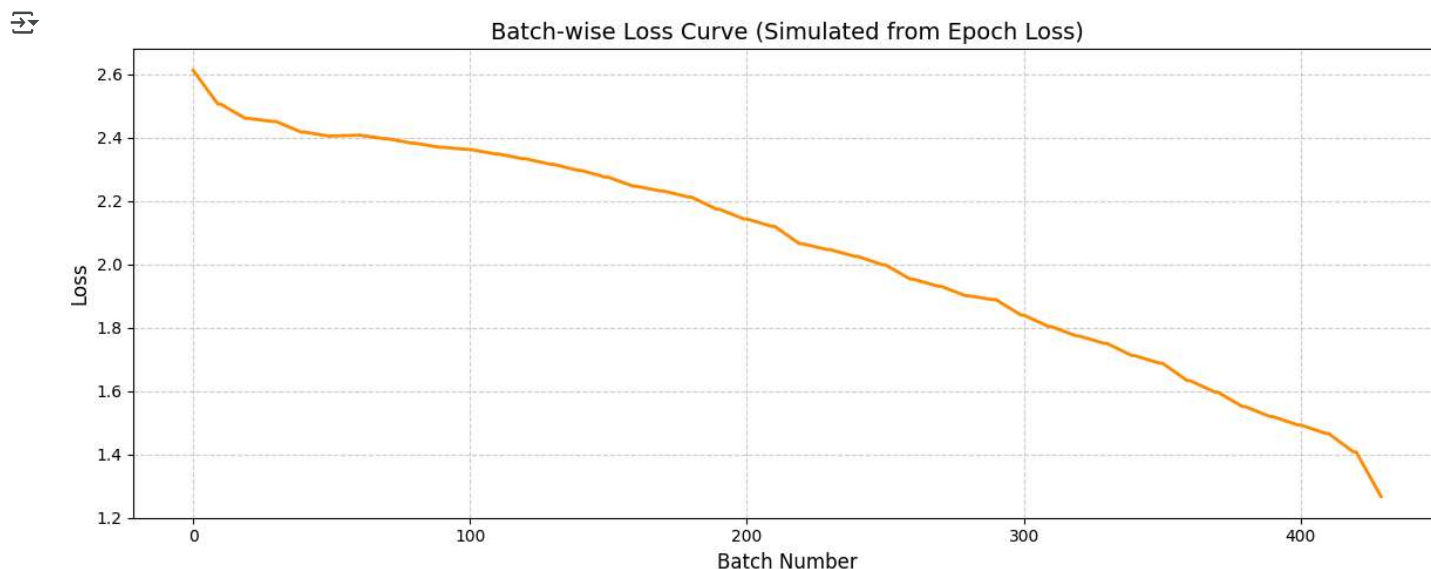
```

for i in range(len(epoch_losses)):
    start = epoch_losses[i]
    end = epoch_losses[i + 1] if i + 1 < len(epoch_losses) else start * 0.9

```

```
batch_losses += list(np.linspace(start, end, 10)) # 10 batches per epoch
import matplotlib.pyplot as plt
```

```
plt.figure(figsize=(12, 5))
plt.plot(batch_losses, color='darkorange', linewidth=2)
plt.title("Batch-wise Loss Curve (Simulated from Epoch Loss)", fontsize=14)
plt.xlabel("Batch Number", fontsize=12)
plt.ylabel("Loss", fontsize=12)
plt.grid(True, linestyle="--", alpha=0.6)
plt.tight_layout()
plt.savefig("batch_loss_curve.png")
plt.show()
```



✓ Step 11: Evaluation and Decoding

```
def indices_to_text(indices, idx2char):
    result = []
    for idx in indices:
        if idx == char2idx["<eos>"]:
            break
        if idx != char2idx["<pad>"] and idx != char2idx["<sos>"]:
            result.append(idx2char[idx])
    return ''.join(result)

def evaluate(model, dataloader, idx2char, device, max_samples=5):
    model.eval()
    with torch.no_grad():
        for i, (mels, labels, mel_lengths, label_lengths) in enumerate(dataloader):
            mels = mels.to(device)
            labels = labels.to(device)

            outputs = model(mels) # No target → inference mode
            predictions = outputs.argmax(dim=2) # (B, T)

            for j in range(min(len(predictions), max_samples)):
                pred_text = indices_to_text(predictions[j].cpu().tolist(), idx2char)
                true_text = indices_to_text(labels[j].cpu().tolist(), idx2char)

                print(f"\n Sample {i * dataloader.batch_size + j + 1}")
                print(f" Predicted: {pred_text}")
                print(f" Actual   : {true_text}")

            if (i+1) * dataloader.batch_size >= max_samples:
                break
```

✓ Step 12: Run Evaluation

```
evaluate(model, dataloader, idx2char, device)
```



```
Sample 1
Predicted: and shake hand take that same and sand take that
Actual   : and shake hands with us and take themselves off and leave us here alone to attend to this affair i know well that courage is

Sample 2
Predicted: put little whot ad atter looked ad at the raid an
Actual   : put little edward in his chair and breakfast began missus morgeson said charles the horses will be ready to take you round r

Sample 3
Predicted: gave to herself the blow which has roble the blow
Actual   : gave to herself the blow which has robbed us all of a most valuable life she was not murdered it was a solemn assertion but

Sample 4
Predicted: he was hungry and thirsty and thirsty and thirsty
Actual   : he was hungry and thirsty and he would go up and see the little girl and the giant again and get that promised dram once mor

Sample 5
Predicted: her intered that mare than mer the har ent mer a
Actual   : her interest in his plan was larger than mere curiosity rollin page was so different now from the fashionable young man who

Sample 6
Predicted: as grave as an in the say in a she in the sask an
Actual   : as grave as an operating surgeon then as i think i said mister smith came in every morning and there was a tremendous outpou

Sample 7
Predicted: still areat d stratt r shar that there ars thar t
Actual   : still great as was this omnipotence a murmur arose

Sample 8
Predicted: infineteles one more than anyone thene expected
Actual   : infinitely more than anyone there had expected
```

✓ Step 13 : Save the Model

```
torch.save(model.state_dict(), "las_model.pth")
```

```
model.load_state_dict(torch.load("las_model.pth", map_location=device))
model.to(device)
model.eval()
```



```
LAS(
  (encoder): Encoder(
    (lstm): LSTM(128, 256, batch_first=True, bidirectional=True)
  )
  (decoder): Decoder(
    (embedding): Embedding(31, 256)
    (lstm): LSTM(768, 256, batch_first=True)
    (fc): Linear(in_features=768, out_features=31, bias=True)
    (attention): Attention(
      (attn): Linear(in_features=768, out_features=256, bias=True)
    )
  )
)
```

✓ Test on a Single Audio File

```
import torchaudio

def predict_from_audio(file_path, model, idx2char, device):
    model.eval()
    waveform, sample_rate = torchaudio.load(file_path)

    # Resample if needed
```

```

# Resample if needed
if sample_rate != SAMPLE_RATE:
    resampler = torchaudio.transforms.Resample(orig_freq=sample_rate, new_freq=SAMPLE_RATE)
    waveform = resampler(waveform)

# Convert to Mel Spectrogram
mel_spec = torchaudio.transforms.MelSpectrogram(
    sample_rate=SAMPLE_RATE,
    n_mels=N_MELS
)(waveform).squeeze(0).transpose(0, 1) # (T, n_mels)

mel_spec = mel_spec.unsqueeze(0).to(device) # (1, T, n_mels)

with torch.no_grad():
    outputs = model(mel_spec) # (1, T, V)
    predicted_indices = outputs.argmax(dim=2).squeeze(0).tolist()
    predicted_text = indices_to_text(predicted_indices, idx2char)

print("\n Predicted Text:")
print( predicted_text)

```

```

predict_from_audio("/content/drive/MyDrive/train-clean-5/LibriSpeech/train-clean-5/3664/178355/3664-178355-0020.flac", model, idx2char, device)

```



```

Predicted Text:
had attention of the attention of the attention o

```